Coursework for Continuous Assessment

Introduction to Deep Learning

Natural Language Processing: Dialog Systems and Chatbots

Assessment type: Coursework for continuous assessment

Weight: 30% of the final grade

Deadline: Friday 22/05/2026 at 23:59 CET

This coursework is part of the continuous assessment component of the course. Students are required to submit their solutions individually. Group submissions are not permitted.

Each submission must include:

- A PDF report containing all written answers, explanations, methodology, results, tables, figures, and examples of chatbot conversations.
- A ZIP archive containing all Python code required to reproduce the results.
- A README file with clear instructions explaining how to run the code.
- A file containing generated chatbot conversations, for example `output_convo.txt`.
- Any configuration files, requirements files, or scripts needed to reproduce the experiments.

All code must be executable, clearly structured, and sufficiently documented. The report must reference the generated outputs and provide concise but critical explanations of the design choices, model behaviour, performance, and limitations.

Problem Description

Dialog systems and chatbots are among the most visible applications of Natural Language Processing. Early chatbots were mostly rule-based, relying on manually designed patterns and handcrafted responses. Modern conversational systems increasingly use deep learning models, including sequence-to-sequence architectures, attention mechanisms, recurrent neural networks, and hybrid rule-based/neural approaches.

In this coursework, you will design, implement, evaluate, and improve a chatbot. The purpose of the assignment is to understand how conversational models are built from the ground up. Therefore, students must not rely on existing large language models or pretrained generative chatbot systems to produce responses.

You may start from a simple sequence-to-sequence model with attention, adapt the provided starter code, or implement your own neural conversational model using NLP tools. Your objective is not only to make the chatbot produce responses, but to critically improve and evaluate its conversational quality.

Your chatbot should take a user utterance as input and generate an appropriate response. An utterance may be a full sentence, multiple sentences, or a short conversational fragment.

The final system should demonstrate improvements over a baseline in terms of response relevance, coherence, context awareness, personality consistency, and user feedback adaptation.

Data

You may use one or more of the following datasets:

- Cornell Movie-Dialogs Corpus;
- public dialogue datasets;
- open-domain conversational datasets;
- Reddit or Twitter conversation datasets, if appropriately cleaned;
- your own small conversational dataset, provided it is anonymised and ethically appropriate;
- a small custom dataset created for testing memory, personality, or feedback behaviour.

The Cornell Movie-Dialogs Corpus is a suitable starting point. However, movie dialogue often contains dramatic, unnatural, or highly stylised responses. Therefore, relying only on this dataset may lead to a chatbot that behaves inconsistently or unnaturally. You are encouraged to use an additional dataset or to construct a small controlled evaluation set.

Your report must clearly specify:

- which dataset or datasets were used;
- the number of training, validation, and test examples;
- how input-response pairs were constructed;
- how the train, validation, and test splits were created;
- any filtering, cleaning, tokenisation, or normalisation steps;
- any ethical or privacy considerations if custom data is used.

Typical preprocessing steps may include:

- lowercasing text;

- tokenising punctuation;
- replacing digits with a special symbol;
- removing rare tokens;
- removing noisy markup or special characters;
- padding or truncating sequences;
- constructing encoder and decoder input pairs;
- removing duplicated or very short conversations;
- filtering inappropriate or low-quality responses.

Libraries and Implementation Guidelines

You are expected to use Python. You may use any of the following libraries:

- PyTorch or TensorFlow for neural network implementation;
- NLTK, spaCy, or similar libraries for text preprocessing;
- NumPy, pandas, and scikit-learn for data handling and evaluation;
- Matplotlib or similar tools for plots and analysis.

The use of Hugging Face Transformers or similar libraries is not allowed for generating chatbot responses. These libraries may only be used for optional auxiliary tasks, such as sentence embeddings, intent classification, semantic similarity, or evaluation, if clearly justified and declared.

You are not required to implement all low-level neural network operations from scratch. However, you are expected to understand the components you use and explain them clearly in your report.

Possible modelling approaches include:

- sequence-to-sequence models;
- attention-based encoder-decoder models;
- recurrent neural networks such as LSTMs or GRUs;
- character-level or subword-level sequence-to-sequence models;
- retrieval-augmented systems where the retrieval and response-selection logic is implemented by the student;
- hybrid rule-based/neural chatbots;
- small Transformer-style encoder-decoder architectures implemented and trained from scratch.

Restrictions on Large Language Models and Pretrained Generative Chatbots

The purpose of this coursework is to build and understand a chatbot from scratch using deep learning methods studied in the course. Therefore, students are not allowed to use a pretrained large language model or an existing pretrained chatbot as the main response-generation system.

In particular, students may not use models or services such as:

- ChatGPT;
- GPT-style pretrained models;
- DialoGPT;
- BlenderBot;
- T5;
- BART;
- LLaMA;
- Mistral;
- Claude;
- Gemini;
- any other pretrained generative language model or chatbot API.

Students must implement and train their own chatbot model using architectures such as:

- sequence-to-sequence models;
- encoder-decoder recurrent neural networks;
- LSTM- or GRU-based models;
- attention-based encoder-decoder models;
- character-level or word-level sequence models;
- simple Transformer architectures implemented and trained by the student, provided they are not pretrained generative models.

The use of pretrained word embeddings, such as GloVe, word2vec, or FastText, is allowed, provided that the response-generation model itself is trained by the student.

BERT-like encoder models may only be used for auxiliary components, such as intent classification, sentence similarity, retrieval of previous conversation turns, or evaluation. They may not be used as the main chatbot response generator.

Any use of external models, pretrained embeddings, code templates, tutorials, or third-party libraries must be clearly declared in the report.

Reference Model

A standard baseline for this assignment is a sequence-to-sequence chatbot with attention. The encoder processes the input utterance and the decoder generates the response. During decoding, the model predicts one token at a time.

A simplified sequence-to-sequence setup can be described as:

```
encoder_outputs, encoder_state = encoder(input_sequence)
decoder_outputs = decoder(target_sequence, encoder_state, attention=True)
```

At inference time, a simple greedy decoding strategy selects the most likely token at each step:

```
next_token = argmax(output_distribution)
```

This greedy strategy often produces repetitive, generic, or unnatural responses. You are expected to improve or compare decoding strategies using at least two of the following methods:

- greedy decoding;
- beam search;
- top-k sampling;
- temperature sampling;
- response filtering;
- diversity-promoting decoding.

Exercises

Exercise 1. Data Preprocessing, Exploration, and Baseline Chatbot (15 marks)

a) Select and preprocess a dialogue dataset suitable for training a chatbot.

b) Construct input-response pairs for training.

c) Perform exploratory analysis of the dataset, including at least:

- number of conversations or input-response pairs;
- vocabulary size;
- average input and response length;
- examples of noisy or problematic samples.

d) Implement or adapt a baseline chatbot model.

e) Train the baseline model and generate sample conversations.

f) Discuss the main weaknesses of the baseline chatbot.

Your discussion should include examples of poor responses, such as irrelevant replies, repetitive answers, generic statements, hallucinated facts, contradictions, or failure to answer simple questions.

Exercise 2. Improved Neural Chatbot and Decoding Strategy (20 marks)

In this exercise, you must improve the baseline chatbot using a stronger architecture, a better decoding strategy, or both. The improved chatbot must be trained by you. You are not allowed to use a pretrained generative chatbot or a large language model as the response generator.

a) Implement at least one substantial model improvement. Examples include:

- attention mechanism;
- deeper encoder-decoder architecture;
- LSTM- or GRU-based encoder-decoder model;
- character-level sequence-to-sequence model;
- word-level sequence-to-sequence model with pretrained word embeddings;
- a small Transformer encoder-decoder model trained from scratch.

b) Compare at least two decoding strategies, for example greedy decoding versus beam search, top-k sampling, or temperature sampling.

c) Provide qualitative examples showing how the decoding strategy affects response quality.

d) Report at least two quantitative metrics, such as:

- validation loss;
- perplexity;
- BLEU, ROUGE, or another text similarity metric;
- response diversity metrics such as distinct-1 and distinct-2.

e) Discuss the limitations of automatic metrics for chatbot evaluation.

Your answer should make clear whether improvements come from the model architecture, training data, decoding strategy, or post-processing.

Exercise 3. Training on Multiple Datasets and Domain Effects (15 marks)

Bots are strongly affected by the data used to train them. A model trained only on movie dialogues may produce dramatic, unrealistic, or inconsistent responses.

a) Train or fine-tune your chatbot using at least one additional dataset, or construct a controlled supplementary dataset for specific conversational behaviours.

b) Explain how you combined, filtered, or selected the datasets.

c) Compare chatbot behaviour before and after using the additional data.

d) Analyse how dataset choice affects tone, vocabulary, response diversity, and conversational quality.

e) Provide examples where the dataset improves the chatbot and examples where it introduces new problems.

You should explicitly discuss dataset bias, domain mismatch, and the effect of training data on chatbot personality and reliability.

Exercise 4. Conversational Memory and Context Handling (20 marks)

A major limitation of simple chatbots is that they only use the most recent user utterance. For example, if a user says their name and later asks "What is my name?", a memoryless chatbot will usually fail.

a) Implement a mechanism that allows the chatbot to remember information from previous turns.

b) Demonstrate that the chatbot can store and reuse simple facts from the conversation.

c) Test the chatbot on examples involving names, preferences, locations, or previous statements.

d) Compare the chatbot with and without memory.

e) Discuss the limitations of your memory mechanism, including failure cases.

You may implement memory using handcrafted rules, a dialogue state, a retrieval mechanism, a context window, embeddings, vector search, or another suitable approach.

BERT-like encoder models, such as BERT, RoBERTa, DistilBERT, Sentence-BERT, MiniLM, or MPNet, may be used only for auxiliary memory retrieval or semantic similarity. They may not be used to generate chatbot responses.

Example:

```
User: My name is Ana.
Bot: Nice to meet you, Ana.
User: What is my name?
Bot: Your name is Ana.
```

Your implementation should handle at least five different memory-based test cases.

Exercise 5. Chatbot Personality and Consistency (15 marks)

The chatbot should not behave like a random mixture of different speakers. It should have a consistent personality, tone, and basic personal information.

a) Define a personality profile for your chatbot.

b) Make the chatbot answer consistently to questions about its name, background, interests, role, or preferences.

c) Implement one method to encourage personality consistency.

d) Provide at least five sample conversations demonstrating the personality.

e) Include at least three adversarial or repeated questions to test whether the personality remains consistent.

f) Discuss whether the personality is produced by the model, rules, conditioning, retrieval, or another mechanism.

Possible approaches include:

- injecting profile information into the encoder or decoder input;
- filtering training data by speaker;
- using handcrafted rules for identity-related questions;
- conditioning the model on a persona description;
- using retrieval from a personality profile.

Exercise 6. User Feedback Loop and Adaptation (15 marks)

A practical chatbot should be improvable through feedback. In this exercise, you will implement a mechanism that allows users to correct bad responses.

a) Implement a feedback loop where the user can indicate that a response is wrong.

b) Allow the user to provide a better response.

c) Store the correction in a structured way.

d) Use the stored feedback to improve future responses.

e) Evaluate the chatbot before and after applying feedback.

Example interaction:

```
User: What is your favourite food?
Bot: I do not know.
User: That's wrong. You should say: I like pizza.
Bot: Thanks, I will remember that.
User: What is your favourite food?
Bot: I like pizza.
```

Your evaluation should include:

- at least five feedback corrections;
- at least five before-and-after examples;
- discussion of whether the feedback changes only exact questions or generalises to related questions;
- discussion of the risks of user feedback, including incorrect, malicious, or contradictory feedback.

Exercise 7. Evaluation, Ablation Study, and Error Analysis (15 marks)

In this final exercise, you must evaluate the complete chatbot system critically.

a) Compare at least three system variants, for example:
   - baseline chatbot;
   - improved model;
   - improved model with memory;
   - improved model with personality;
   - improved model with feedback loop.

b) Report quantitative metrics where appropriate.

c) Provide qualitative examples of successful and unsuccessful conversations.

d) Conduct an ablation study showing the effect of removing one component, such as memory, feedback, personality conditioning, or improved decoding.

e) Provide an error analysis with at least five failure cases.

f) Discuss ethical and safety limitations of your chatbot.

Your error analysis should identify specific types of failure, such as:

- irrelevant responses;
- contradictions;
- repetitive outputs;
- unsafe or inappropriate responses;
- failure to remember context;
- excessive generic answers;
- hallucinated facts.

Optional Extensions

The following extensions are optional but may improve the quality of your submission:

- Implement a character-level sequence-to-sequence model.

- Implement a small Transformer-based sequence-to-sequence model trained from scratch.
- Use beam search, top-k sampling, or temperature sampling for decoding.
- Add sentiment-aware responses.
- Add retrieval from previous conversations.
- Add safety filters for inappropriate or offensive responses.
- Compare neural, rule-based, and hybrid chatbot strategies.
- Add a simple user interface.
- Implement vector-based semantic memory.
- Evaluate the chatbot with human ratings from classmates or friends.

Optional extensions must still respect the restriction that pretrained generative language models and existing chatbot APIs may not be used as the response-generation engine.

Expected Deliverables

Your final submission must include:

1. A PDF report.
2. A ZIP file containing all code.
3. A README file explaining how to run your code.
4. A file containing generated chatbot conversations, for example `output_convo.txt`.
5. A clear description of the datasets used.
6. A clear explanation of each improvement implemented.
7. Tables or plots showing training and evaluation results.
8. A short ablation study.
9. A section discussing limitations, ethical issues, and possible future improvements.

The report should include:

- introduction and motivation;
- dataset description and preprocessing;

- exploratory data analysis;
- baseline model architecture;
- improved model architecture;
- training procedure;
- decoding strategy;
- implemented improvements;
- memory and personality design;
- feedback loop design;
- sample conversations;
- quantitative evaluation;
- qualitative evaluation;
- ablation study;
- error analysis;
- limitations and possible future improvements.

Code Requirements

All code should be written in Python and should have a logical structure. You should:

- organise code into functions or classes;
- avoid unnecessary duplication;
- comment important parts of the code;
- include clear instructions for running the project;
- ensure that paths and dependencies are documented;
- include any required configuration files;
- include a `requirements.txt` or equivalent environment file;
- make the main experiment reproducible from the README instructions.

The submitted code must be your own work. If you use external libraries, starter code, pretrained embeddings, datasets, tutorials, or online examples, these must be clearly acknowledged.

Assessment Criteria

Marks will be awarded based on:

- correctness and clarity of the implementation;
- quality of data preprocessing and dataset analysis;
- quality of the baseline chatbot;
- technical quality of the improved model;
- quality of the decoding comparison;
- effectiveness of conversational memory;
- consistency of chatbot personality;
- quality of the feedback loop;
- strength of the evaluation and ablation study;
- depth of error analysis;
- clarity of the written report;
- quality of code structure and documentation;
- critical discussion of limitations and ethical issues.

Marking Scheme

Component	**Marks**
Data preprocessing, exploration, and baseline chatbot	15
Improved neural model and decoding strategy	20
Multiple datasets and domain effects	15
Conversational memory and context handling	20
Chatbot personality and consistency	15
User feedback loop and adaptation	15
Evaluation, ablation study, and error analysis	15
Code quality, reproducibility, and report clarity	10
Total	**125**

The final mark will be normalised to 30% of the overall course grade.